Distributed Lupus in Fabula

Final Report for the Distributed Systems Course 2025/2026

Jacopo Bonifazi
author1@email.it

Alexandru Nicolescu
author2@gmail.com

Leonardo Vorabbi
leonardo.vorabbi@studio.unibo.it

July 10, 2026

Distributed Lupus in Fabula is a real-time multiplayer web application that brings the classic social deduction game Lupus in Fabula online, allowing groups of players to create or join a game lobby from any browser and play a full match together over the internet. The system is composed of a Vue 3 single-page frontend and a FastAPI/Socket.IO backend, communicating via REST for stateless operations and via WebSocket for real-time, low-latency interactions such as votes, phase transitions, and private role actions. Game and lobby state is kept in Redis rather than a traditional database, acting both as a shared source of truth across backend replicas and as a Pub/Sub broker that lets events generated on one instance reach clients connected to any other instance. This design allows the backend to run as multiple concurrent, interchangeable replicas behind a reverse proxy, supporting many simultaneous matches while keeping every player's view of the game synchronized in real time.

Disclaimer

During the preparation of this work, the authors used LLMs to to refine syntax and correct linguistic errors.

After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the final report/artifact

1 Concept

1.1 Type of Product

The project is a web application, specifically a real-time multiplayer game inspired by the traditional social deduction game *Lupus in Fabula*. It is a full-stack distributed system composed of:

- A **frontend**: a Vue 3 Single Page Application (SPA), accessible via browser.
- A **backend**: a set of stateless, horizontally replicated web services built with FastAPI and Socket.IO, responsible for game logic, session management, and real-time event propagation.
- A **shared data layer**: Redis, used both as a low-latency state store and as a Pub/Sub message broker connecting the backend replicas.

The whole system is designed to be deployed as a multi-container distributed application, rather than as a single monolithic server.

1.2 Use Case Description

What the software does. The application lets a group of players create or join a virtual "lobby," wait until everyone is ready, and then play a phase-based game of hidden roles and social deduction. Once a match starts, the game state moves automatically through a fixed sequence of phases:

$$\textbf{LOBBY} \rightarrow \textbf{DAY} \rightarrow \textbf{VOTING} \rightarrow \textbf{NIGHT} \rightarrow \cdots \rightarrow \textbf{ENDED}$$

driven by server-side timers.

During DAY/VOTING, players discuss and vote to eliminate a suspect; during NIGHT, players with special roles perform hidden actions that are resolved by the server before the next day begins. The match ends automatically once a win condition is met.

Where the users are. Players are geographically dispersed (or at least not necessarily co-located): each player connects independently from their own browser, over the public Internet, to a shared game session identified by a room/lobby code.

When and how frequently they interact. Interaction is continuous and real-time for the whole duration of a match (several minutes to tens of minutes), not just at fixed request/response intervals. Players must react within phase time limits (e.g., voting windows, night-action windows), so the system needs low-latency, event-driven communication rather than simple periodic polling.

How they interact. All interaction happens through a web browser via the Vue SPA. The frontend communicates with the backend in two complementary ways:

- **REST** calls for stateless operations such as creating a lobby, joining a lobby, or fetching the current game state.
- **WebSocket** for real-time, bidirectional events: votes, phase changes, chat, private role actions, disconnect/reconnect notifications, etc.

Does the system store user data? Which, and where? The system does not use a persistent relational database for user accounts since there is no long-term profile storage. Instead, it relies on Redis as a shared, TTL-based store for all session-scoped game data:

- room/lobby state and configuration;
- the list of connected players and their ready status;
- assigned roles (Wolf, Seer, Villager);
- votes cast during the DAY/VOTING phase and private night actions;
- phase timers and current phase;
- host identity.

On the client side, the browser itself retains a minimal amount of session data in local storage, purely to support reconnection and continuity of a player's session across page reloads or brief disconnects.

Roles. The system distinguishes several actor roles at different levels:

- *Application-level roles*: **Villager**, **Wolf**, **Seer**, each with different permitted actions and visibility (e.g., Wolves share a private night chat, the Seer can inspect one player per night).
- *Session-level roles*: the **Host**, who configures the match, can remove players from the lobby, and starts the game; versus **regular players**, who can only join, ready up, vote, and act according to their assigned role.
- *System role*: the backend itself acts as an autonomous **orchestrator**. It owns the authoritative game state, advances phases via timers, resolves votes and night actions, and enforces which actions are legal at any given moment.

1.3 Why Distribution Is Needed

Distribution in this project is not motivated by geographic proximity to users, but primarily by scalability, real-time synchronization, and fault tolerance:

- **Horizontal scalability / resource sharing.** The backend is designed to run as multiple stateless replicas behind a reverse proxy, so that many concurrent lobbies and matches can be served in parallel without a single process becoming a bottleneck. Because game state lives in Redis rather than in each process's memory, any replica can handle requests for any room, allowing the system to scale out simply by adding more backend instances.
- **Shared state across replicas.** Since each backend instance is stateless with respect to game data, Redis acts as the single source of truth for lobby/game state. This is what makes horizontal scaling actually work: a player's WebSocket connection can be handled by one replica while another replica processes a REST call for the same match, and both see a consistent view of the state.
- **Cross-replica real-time propagation.** Because clients belonging to the same room can be connected to different backend replicas, a simple in-process event emitter is not sufficient. The system uses Redis Pub/Sub to broadcast game events (vote updates, phase transitions, chat messages, etc.) across all replicas, ensuring that every client in a room receives updates regardless of which instance generated them or which instance is handling their socket.
- **Fault tolerance and continuity.** The backend includes explicit mechanisms for reconnecting to Redis, retrying failed operations, sweeping/recovering timers, and reconciling state after temporary inconsistencies (anti-entropy). This matters because a real-time game cannot simply crash or stall if a single backend replica fails, a Redis connection drops momentarily, or a player's browser disconnects and reconnects mid-match. The match state and timers must survive these events.
- **Reduced dependency on a single instance.** Because the game state is decoupled from any specific process, a backend replica can be restarted, replaced, or scaled down/up without necessarily terminating ongoing matches, and a reconnecting player can resume their session against a different replica than the one they were originally connected to.

In summary, this system is distributed primarily to support many concurrent matches, keep real-time state synchronized across independent backend replicas, and remain resilient to instance failures and client disconnections.

2 Requirements Elicitation and Analysis

- The requirements must explain **what** (not how) the software being produced should do.

  - you should not focus on the particular problems, but exclusively on what you want the application to do.

- Requirements must be clearly identified, and possibly numbered
- Requirements are divided into:
  - **Functional**: some functionality the software should provide to the user
  - **Non-functional**: requirements that do not directly concern behavioural aspects, such as consistency, availability, etc.
  - **Implementation**: constrain the entire phase of system realization, for instance by requiring the use of a specific programming language and/or a specific software tool
    * these constraints should be adequately justified by political / economic / administrative reasons...
    * ... otherwise, implementation choices should emerge *as a consequence of* design
- If there are domain-specific terms, these should be explained in a glossary
- Each requirement must have its own **acceptance criterion**
  - these will be important for the validation phase

2.1 Relevant Distributed System Features

Motivate which distributed system features are relevant for your project, and which are not.

- transparency
  - does your system need to hide distribution details from users or developers?
  - is it important that failures, location, or replication are invisible?
- fault tolerance, dependability – availability, reliability, integrity, maintainability, safety
  - what happens if a component fails? is uninterrupted service required?
  - is data loss or corruption unacceptable?
  - how quickly must the system recover from faults?
- scalability
  - will the system need to handle increasing numbers of users, requests, or data?
  - is it expected to grow over time?
- security, trust

  - is sensitive data being processed or stored?
  - are there multiple user roles with different permissions?
  - is authentication or authorization required?
- resource sharing
  - do multiple users or components need access to shared resources?
  - is coordination or synchronization needed?
- openness, interoperability, heterogeneity of components
  - Will your system interact with external systems or use components built with different technologies?
  - Is standardization or compatibility important?
- evolvability, maintainability
  - Will the system need to be updated or extended after deployment?
  - Is long-term maintenance a concern?
- performance, concurrency, computation / communication efficiency, bandwidth
  - Are there strict requirements on response time or throughput?
  - Will many operations happen in parallel?
  - Is network usage a concern?
- economy, costs
  - Are there budget constraints for development, deployment, or operation?
  - Is minimizing resource usage important?

3 Design

This chapter explains the strategies used to meet the requirements identified in the analysis. Ideally, the design should be the same, regardless of the technological choices made during the implementation phase.

> You can re-arrange the sections as you prefer, but all the sections must be present in the end

Important: try to motivate your design choices in relation to the requirements and features identified in section 2 and section 2.1.

3.1 Architecture

- Which architectural style?
  - why?

3.2 Infrastructure

- are there *infrastructural components* that need to be introduced? *how many*?
  - e.g. *clients*, *servers*, *load balancers*, *caches*, *databases*, *message brokers*, *queues*, *workers*, *proxies*, *firewalls*, *CDNs*, *etc.*
- how do components *distribute* over the network? *where*?
  - e.g. do servers / brokers / databases / etc. sit on the same machine? on the same network? on the same datacenter? on the same continent?
- how do components *find* each other?
  - how to *name* components?
  - e.g. DNS, *service discovery*, *load balancing*, *etc.*

Component diagrams are welcome here

3.3 Modelling

- which **domain entities** are there?
  - e.g. *users*, *products*, *orders*, *etc.*
- how do *domain entities* **map to** *infrastructural components*?
  - e.g. state of a video game on central server, while inputs/representations on clients
  - e.g. where to store messages in an IM app? for how long?
- which **domain events** are there?
  - e.g. *user registered*, *product added to cart*, *order placed*, *etc.*
- which sorts of **messages** are exchanged?
  - e.g. *commands*, *events*, *queries*, *etc.*
- what information does the **state** of the system comprehend
  - e.g. *users' data*, *products' data*, *orders' data*, *etc.*

Class diagram are welcome here

3.4 Interaction

- how do components *communicate*? *when*? *what*?
- *which* **interaction patterns** do they enact?

Sequence diagrams are welcome here

3.5 Behaviour

- how does *each* component **behave** individually (e.g. in *response* to *events* or messages)?
  - some components may be *stateful*, others *stateless*
- which components are in charge of updating the **state** of the system? *when*? *how*?

  State diagrams are welcome here

3.6 Data and Consistency Issues

- Is there any data that needs to be stored?
  - *what* data? *where*? *why*?
- how should *persistent data* be **stored**?
  - e.g. relations, documents, key-value, graph, etc.
  - why?
- Which components perform queries on the database?
  - *when*? *which* queries? *why*?
  - concurrent read? concurrent write? why?
- Is there any data that needs to be shared between components?
  - *why*? *what* data?

3.7 Fault-Tolerance

- Is there any form of data **replication** / federation / sharing?
  - *why*? *how* does it work?
- Is there any **heart-beating**, **timeout**, **retry mechanism**?
  - *why*? *among* which components? *how* does it work?
- Is there any form of **error handling**?
  - *what* happens when a component fails? *why*? *how*?

3.8 Availability

- Is there any **caching** mechanism?
  - *where*? *why*?
- Is there any form of **load balancing**?
  - *where*? *why*?

- In case of **network partitioning**, how does the system behave?
  - *why*? *how*?

3.9 Security

- Is there any form of **authentication**?
  - *where*? *why*?
- Is there any form of **authorization**?
  - which sort of *access control*?
  - which sorts of users / *roles*? which *access rights*?
- Are **cryptographic schemas** being used?
  - e.g. token verification,
  - e.g. data encryption, etc.

4 Implementation

Please report here all the implementation (technology-dependent) choices you made while implementing your design.

> If you run out of time for this project, you may consider leaving some aspect unimplemented, and simply discuss how you would have implemented them. In this case, better would be to discuss unimplemented features in section 9.

- which **network protocols** to use?
  - e.g. UDP, TCP, HTTP, WebSockets, gRPC, XMPP, AMQP, MQTT, etc.
- how should *in-transit data* be **represented**?
  - e.g. JSON, XML, YAML, Protocol Buffers, etc.
- how should *databases* be **queried**?
  - e.g. SQL, NoSQL, etc.
- how should components be *authenticated*?
  - e.g. OAuth, JWT, etc.
- how should components be *authorized*?
  - e.g. RBAC, ABAC, etc.

4.1 Technological details

- any particular *framework* / *technology* being exploited goes here

5 Validation

5.1 Automatic Testing

- how were individual components ***unit*-test**ed?
- how was communication, interaction, and/or integration among components tested?
- how to ***end-to-end*-test** the system?
  - e.g. production vs. test environment
- for each test specify:
  - rationale of individual tests
  - how were the test automated
  - how to run them
  - which requirement they are testing, if any

  recall that *deployment* **automation** is commonly used to *test* the system in *production-like* environment

  recall to test corner cases (crashes, errors, etc.)

5.2 Acceptance test

- did you perform any *manual* testing?
  - what did you test?
  - why wasn't it automatic?

6 Deployment

- should one install your software from scratch, how to do it?
  - provide instructions
  - provide expected outcomes
- what software should be installed on the machines to run your project? which versions?
- should one set environment variables or configuration files?
- if you're using containerization (e.g. Docker), describe how you engineered your deployment scrips (e.g. `docker-compose.yml` files)

7 User Guide

- how to use your software?
  - provide instructions
  - provide expected outcomes
  - provide screenshots if possible

8 Self-evaluation

- An individual section is required for each member of the group
- Each member must self-evaluate their work, listing the strengths and weaknesses of the product
- Each member must describe their role within the group as objectively as possible.

It should be noted that each student is only responsible for their own section.

9 Future works

Discuss possible future works, improvements, extensions, optimizations, etc.

Also discuss here any unimplemented feature, and how you would have implemented it.

References